

Ejercicio 1- Overloading y relaciones entre clases
Análisis y diseño de 2027 FIFA Women's World Cup

Ejercicio

Análisis

Cree una tabla en la que se incluya toda la información solicitada. Puede hacer una tabla para propiedades y otra para métodos:

1. ¿Qué propiedades y métodos tendrá cada clase?
2. ¿Qué tipo de datos deben tener las propiedades y métodos de cada clase?
3. ¿Cuáles deben ser los modificadores de visibilidad de los miembros en cada clase?
4. ¿Qué parámetros serán requeridos por los métodos en sus clases?
5. ¿Qué propósito tendrá dentro de la clase?

Clase	MVC	Motivo
SistemaVentas	Modelo	Contiene toda la lógica del algoritmo de compra.
Comprador	Modelo	Representa al comprador actual.
Ticket	Modelo	Representa la solicitud de compra y su número aleatorio.
Localidad	Modelo	Representa cada localidad del estadio.
VistaMundial	Vista	Entrada y salida de datos.
ControladorMundial	Controlador	Coordina la vista y el modelo.
Main	Driver Program	Punto de entrada del programa.

Atributos (Propiedades)

#	Atributo (nombre)	Tipo de dato	Visibilidad	Valor inicial	Propósito/Descripción
clase: Localidad rol (MVC): Modelo					
1.	nombreLocalidad	String	Private	“ ”	Identifica la localidad (localidad_1, localidad_5 o localidad_10).
2.	precio	double	Private	0.0	Precio de cada boleto en esta localidad.
3.	capacidadMaxima	int	Private	20	Cantidad máxima de boletos que puede vender la localidad.
4.	boletosVendidos	int	Private	0	Cantidad de boletos vendidos hasta el momento.
Clase: Ticket rol (MVC): Modelo					
1.	numeroAleatorio	int	Private	0	Número aleatorio único generado para identificar el ticket del comprador.
2.	valido	boolean	Private	False	Indica si el ticket continúa siendo válido durante el proceso de compra.
3.	localidadAsignada	Localidad	Private	null	Referencia a la localidad asignada al ticket.
Clase: Comprador rol (MVC): Modelo					
1.	nombreComprador	String	Private	“ ”	Almacena el nombre completo del comprador.
2.	email	String	Private	“ ”	Almacena el correo electrónico del comprador.
3.	cantidadBoletos	int	Private	0	Cantidad de boletos que el comprador desea adquirir.

4.	presupuestoMaximo	double	Private	0.0	Presupuesto máximo que el comprador está dispuesto a pagar por la compra.
5.	ticket	Ticket	Private	null	Referencia al ticket asignado al comprador durante el proceso de compra.
Clase: SistemaVentas rol (MVC): Modelo					
1.	compradorActual	Comprador	Private	null	Referencia al comprador activo del sistema.
2.	localidad1	Localidad	Private	null	Representa la localidad 1 del estadio.
3.	localidad5	Localidad	Private	null	Representa la localidad 5 del estadio.
4.	localidad10	Localidad	Private	null	Representa la localidad 10 del estadio.

Métodos (Comportamiento)

#	Método (nombre)	Tipo de retorno	Parámetros	Visibilidad	Propósito/Responsabilidad
clase: Localidad rol (MVC): Modelo					
1.	getNombreLocalidad()	String		public	Devuelve el nombre de la localidad.
2.	getPrecio()	Double		Public	Devuelve el precio del boleto
3.	getBoletosVendidos()	Int		Public	Devuelve los boletos vendidos.
4.	getBoletosDisponibles()	Int		Public	Calcula y devuelve los boletos disponibles.
5.	tieneEspacio()	Boolean		Public	Indica si aún existe al menos un boleto disponible
6.	venderBoletos()	Void	(Int cantidad)	Public	Incrementa la cantidad de boletos vendidos según la compra realizada.
Clase: Ticket rol (MVC): Modelo					
1.	getNumeroAleatorio()	int		public	Devuelve el número del ticket
2.	isValido()	boolean		Public	Devuelve el estado actual de validez del ticket.
3.	getLocalidadAsignada()	Localidad		Public	Devuelve la localidad asignada al ticket.
4.	setLocalidadAsignada()	void	(Localidad localidad)	Public	Asigna la localidad correspondiente al ticket.
Clase: Comprador rol (MVC): Modelo					
1.	getNombreComprador()	String		public	Devuelve el nombre del comprador

2.	getEmail()	String		Public	Devuelve el correo electrónico del comprador
3.	getCantidadBoletos()	Int		Public	Devuelve la cantidad de boletos solicitados
4.	getPresupuestoMaximo()	Double		Public	Devuelve el presupuesto máximo del comprador
5.	getTicket()	Ticket		Public	Devuelve el ticket asociado al comprador
6.	setTicket()	Void	(Ticket Ticket)	Public	Asigna un ticket al comprador

Clase: SistemaVentas | rol (MVC): Modelo

1.	generarTicket()	void		Public	Genera y asigna un número aleatorio al ticket del comprador actual.
2.	validarTicket()	boolean		Public	Genera los números aleatorios del rango, valida el ticket y actualiza su estado.
3.	asignarLocalidad()	void		Public	Selecciona aleatoriamente una localidad y la asigna al ticket.
4.	validarEspacio()	boolean		Public	Comprueba si la localidad tiene disponibilidad suficiente.
5.	validarPresupuesto()	boolean		Public	Verifica que el presupuesto del comprador cubra el costo de los boletos.
6.	venderBoletos()	void		Public	Registra la venta en la localidad correspondiente.
7.	consultarDisponibilidadTotal()	String		Public	Devuelve la disponibilidad de las tres localidades.
8.	consultarDisponibilidadLocalidad()	String	(String nombreLocalidad)	Public	Devuelve la disponibilidad de una localidad específica.

9.	generarReporteCaja()	String		Public	Calcula el dinero recaudado por todas las localidades.
10.	crearComprador()	void	(String, nombreComprador, String email, int cantidadBoletos, double presupuestoMaximo, Ticket ticket)	Public	Crea/Reemplaza la instancia actual del comprador activo.
11.	getCompradorActual()	String		Public	Devuelve al comprador actual.